

Conceptos básicos

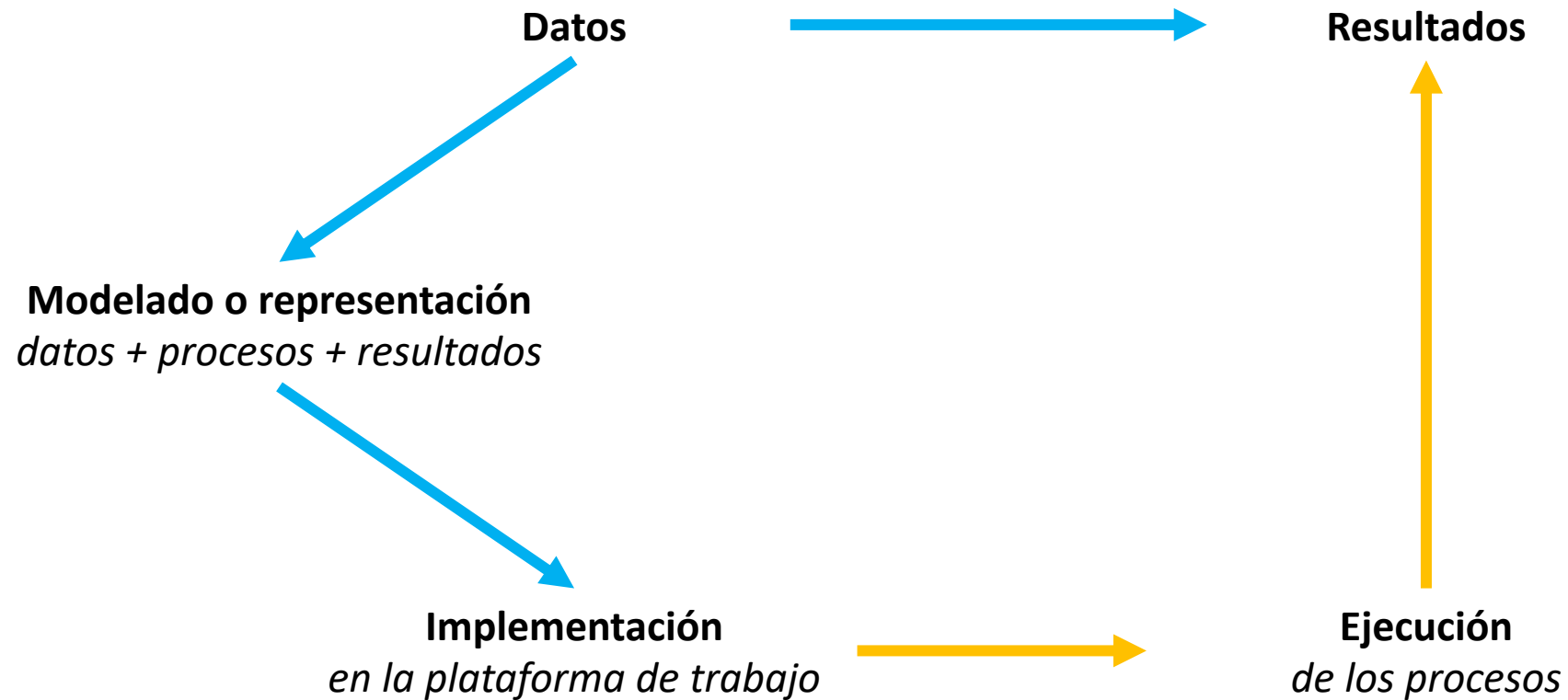
Índice

- Algoritmos
- Programas
- Lenguajes de programación
- Traductores
- Descripción de los lenguajes
- Estructura de un programa
- Elementos de un programa
- Salida y Entrada
- Tipos de datos y operadores
- Depuración de programas

Algoritmos

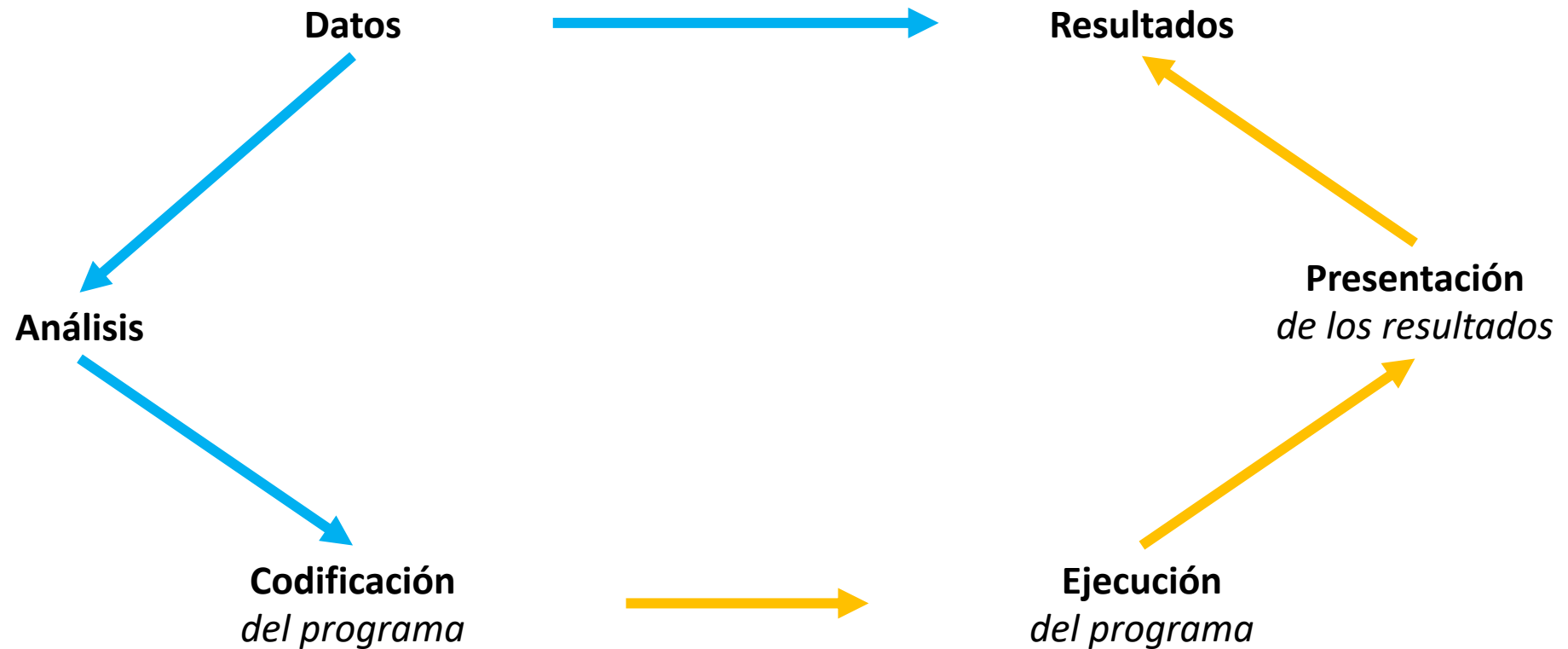
Algoritmos. Conceptos

¿Cómo se resuelve un problema en general?



Algoritmos. Conceptos

¿Cómo se resuelve un problema en un computador?



Algoritmos. Conceptos

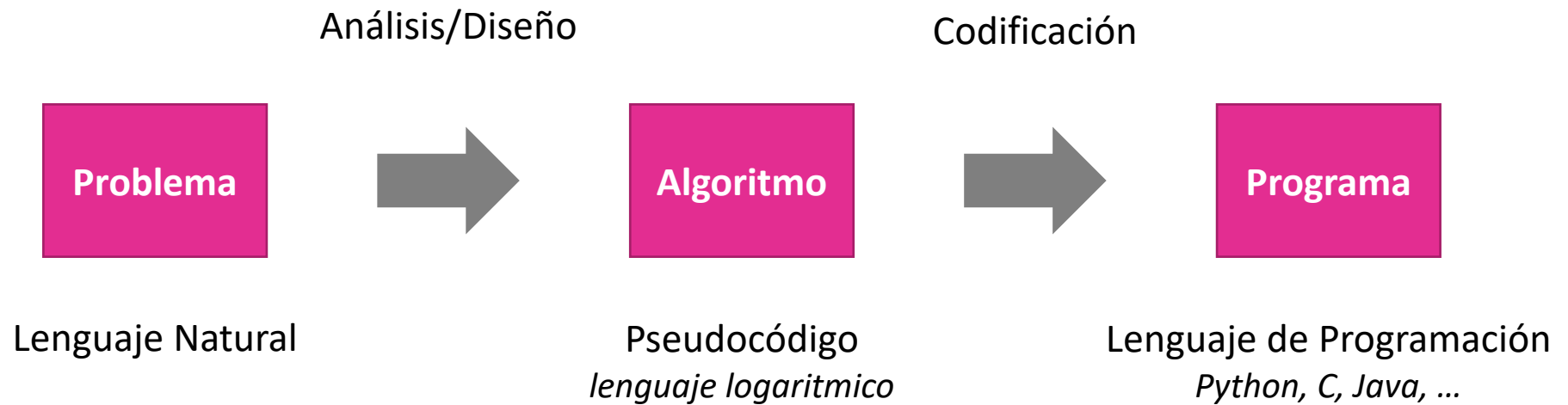
- Problemas que se tratan de resolver en programación
 - Cómputo o cálculo
 - Gestión comercial
 - Control
 - Tratamiento de la señal
 - Lúdicos
 - Inteligencia Artificial
 - E-commerce
 - Etc
- Problemas cuya solución puede ser llevada a cabo por un computador
 - Herramienta que permite tratar de forma automática problemas de tratamiento de la información

Algoritmos. Conceptos

- El objetivo de la programación es poder construir programas que puedan resolver problemas
- ¿Qué es un programa?
 - Es un conjunto de instrucciones que se ejecutan en una máquina para llevar a cabo una determinada tarea.
 - Los lenguajes de programación están diseñados para que los programadores indiquen a una máquina de manera explícita las tareas a realizar

Algoritmos. Conceptos

¿Cómo se realiza un programa?



Algoritmo. Conceptos

Algoritmo: Conjunto ordenado y finido de pasos que especifican, de forma precisa y sin ambigüedades, la sentencia de operaciones que se han de realizar para la resolución de un problema

Para calcular el factorial de un número n basta con multiplicar todos los números enteros positivos entre ese número n y el 1.

Lenguaje Natural
Solución informal

- 1) $i = 1$
- 2) $f = 1$
- 3) si $i > n$ ir al paso 7
- 4) $i = i + 1$
- 5) $f = f * i$
- 6) ir al paso 3
- 7) factorial = f

Lenguaje Algorítmico
Pseudocódigo

Algoritmo. Conceptos

Programa: Codificación en cualquier lenguaje de programación de uno o varios algoritmos

```
1)  i = 1
2)  f = 1
3)  si i > n ir al paso 7
4)  i = i + 1
5)  f = f * i
6)  ir al paso 3
7)  factorial = f
```

Lenguaje Algorítmico
Pseudocódigo

```
i = 1
f = 1
while i < n:
    i = i + 1
    f = f * i
factorial = f
```

Programa
Lenguaje Python

Algoritmos. Propiedades

- Condiciones necesarias
 - Finitud
 - No ambigüedad
- Propiedades deseables
 - Generalidad
 - Eficiencia
 - Independencia de la máquina

Algoritmos. Representación

- Diagramas de flujo
- Pseudocódigo
- Lenguaje natural
- Lenguajes formales matemáticos

Algoritmos. Representación

- Diagramas de flujo
- Pseudocódigo
- Lenguaje natural
- Lenguajes formales matemáticos



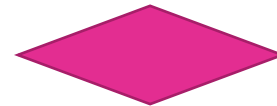
Terminal: Inicio o Fin



Flujo de control



Instrucción



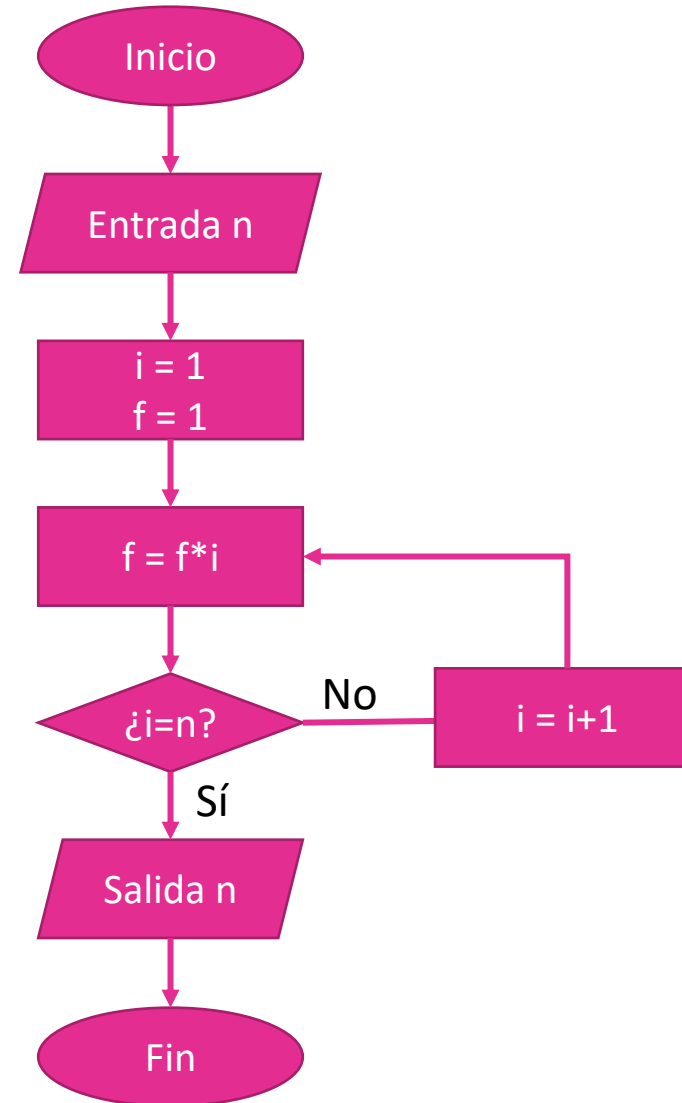
Decisión



Entrada/Salida de datos

Algoritmos. Representación

- Diagramas de flujo
- Pseudocódigo
- Lenguaje natural
- Lenguajes formales matemáticos



Algoritmos. Representación

- Diagramas de flujo
- Pseudocódigo
- Lenguaje natural
- Lenguajes formales matemáticos

- 1) Leer un valor numérico y asignarlo a n
- 2) $i = 1$
- 3) $f = 1$
- 4) $f = f * i$
- 5) Si $i = n$ ir al paso 8 sino ir al paso 6
- 6) $i = i + 1$
- 7) volver al paso 4
- 8) Mostrar el valor de f

Algoritmos. Representación

- Diagramas de flujo
- Pseudocódigo
- Lenguaje natural
- Lenguajes formales matemáticos

Para calcular el factorial de un número n se debe multiplicar todos los números enteros que existen entre el 1 y n .

Algoritmos. Representación

- Diagramas de flujo
- Pseudocódigo
- Lenguaje natural
- Lenguajes formales matemáticos

$$f(n) = \begin{cases} n, & \text{en otro caso} \\ n * f(n - 1), & \text{si } n > 1 \end{cases}$$

Programas

Programas. Sistema Operativo

- Sistema Operativo

- Conjunto de programas que tienen como objetivo facilitar la utilización del computador
 - Acceso a los usuarios autorizados
 - Seguridad y protección
 - Configuración y ejecución de programas
 - etc
- Gestión adecuada de la máquina
 - Gestión de memoria
 - Gestión de periféricos
 - Asignación de uso de recursos de cómputo
 - etc

Programas. Entorno de desarrollo

- Programa cuya finalidad es facilitar el desarrollo de programas utilizando uno varios lenguajes de programación
- Integra herramientas
 - Edición de código fuente (Python, Javascript, Ruby, C, R, ...)
 - Compilación de programas
 - Depuración de programas
 - Ejecución y prueba de programas

Lenguajes de programación

Lenguajes de programación. Tipos

- Lenguaje máquina
- Lenguaje ensamblador
- Lenguajes de alto nivel
 - Lenguaje compilado
 - Lenguaje interpretados
 - Lenguajes híbridos

Lenguajes de programación. Ejecución

- Ejecución interpretada del programa fuente
 - Se itera sobre los siguientes pasos
 - Obtención de la siguiente sentencia del código fuente a ejecutar
 - Análisis de la sentencia y determinación de las acciones a ejecutar
 - Ejecución de las correspondientes acciones
- Ejecución de un programa compilado
 - A partir del programa fuente se realiza la compilación (generar una versión equivalente del programa en lenguaje máquina). Esto se conoce como programa objeto.
 - El programa objeto se puede unir con otros subprogramas para obtener el programa ejecutable: esto lo realiza el montador de enlaces (*linker*)

Lenguajes de programación. Propiedades

- Claridad
- Buena definición
- Notación estándar
- Nivel de abstracción
- Permite programar sin errores
- Verificación de programas

Lenguajes de programación. Paradigmas

- Imperativa
 - C, Pascal, Fortran...
- Orientado a objetos
 - Java, C++, C#...
- Funcional
 - LISP, Scheme...
- Declarativa
 - PROLOG, OCAML...
- Tiempo real
 - VHDL

Lenguajes de programación. Ejemplos

- **Fortran** (1965): **F**ormula **T**ranslator. Cálculo científico.
- **COBOL** (1959): **C**ommon **B**usiness-**O**riented **L**anguage. Gestión.
- **ALBOL** (1960): **A**lgorithmic **L**anguage. Primer intento de formalizar la resolución de problemas. Programación estructurada con tipos de datos.
- **LISP** (1960): **L**ist **P**rocessor. Tratamiento simbólico de la información.
- **BASIC** (1965): **B**eginners **A**ll-**P**urpose **S**ymbolic **I**nstruction **C**ode. Lenguaje interpretado, sin énfasis en tipos de datos.
- **Pascal** (1971). Desciende de ALGOL. Paradigma de la programación estructurada.

Lenguajes de programación. Ejemplos

- **C** (1971): Desarrollado por Dennis Ritchie de AT&T Bell Laboratorios para ofrecer un lenguaje de alto nivel para programar UNIX.
- **Lenguajes orientados a objetos** (C++ 1983, Eiffel 1986, C# 2000): a la hora de organizar el software se focalizan en las abstracciones de datos en lugar de en la funcionalidad.
- **HTML** (1991): Lenguaje de marcado para el desarrollo de páginas web, utilizado para dar formato y estructura a la información en forma de texto e imágenes.
- **Python** (1991): Lenguaje de propósito general centrado en la legibilidad del código que soporta múltiples paradigmas (imperativo, funcional, orientado a objetos ...).
- **Java** (1995): Lenguaje orientado a objetos que se puede ejecutar en cualquier máquina, gracias al concepto de máquina virtual (filosofía *write once, run anywhere*)
- **Dart** (2011). Lenguaje de programación web en código abierto desarrollado por *Google*[®], con el objetivo de sustituir a javascript en el desarrollo web.

Origen del Lenguaje Python

- Python fue desarrollado por Guido Van Rossum en Centro para las Matemáticas y la Informática (Países Bajos)
- Surgió como sucesor del lenguaje ABC, capaz de lanzar excepciones e interactuar con el sistema operativo Amoeba.
- En 1991 se publicó la versión 0.9.0 que ya contaba con manejo de excepciones y tipos modulares como: str, list, dict entre otros.
- En 1994 alcanzó la versión 1.0 donde se le añadieron herramientas de la programación funcional: lambda, reduce, filter y map (cortesía de Amrit Perm, un entusiasta de LISP)
- En 2000 se liberó la versión 2.0 que incluía características importantes del lenguaje Haskell (como la generación de listas). También introdujo un sistema de recolección de basura capaz de detectar las referencias cíclicas

Origen del Lenguaje Python

- En 2008 se liberó la versión 3.0 el cual estaba diseñado para corregir fallos del diseño del lenguaje (cambios que no permitían la compatibilidad hacia atrás con las versiones 2.x).
- Las versiones de Python 2.x y 3.x estaban planteadas para coexistir, y de hecho se lanzaron varias versión en paralelo. Las versiones de Python 2.x y Python 3.x no son compatibles (un programa desarrollado en una versión de Python 2.x puede no ejecutarse con las versiones de Python 3). Existen herramientas como “2to3: para traducir el código de Python 2 a Python 3.

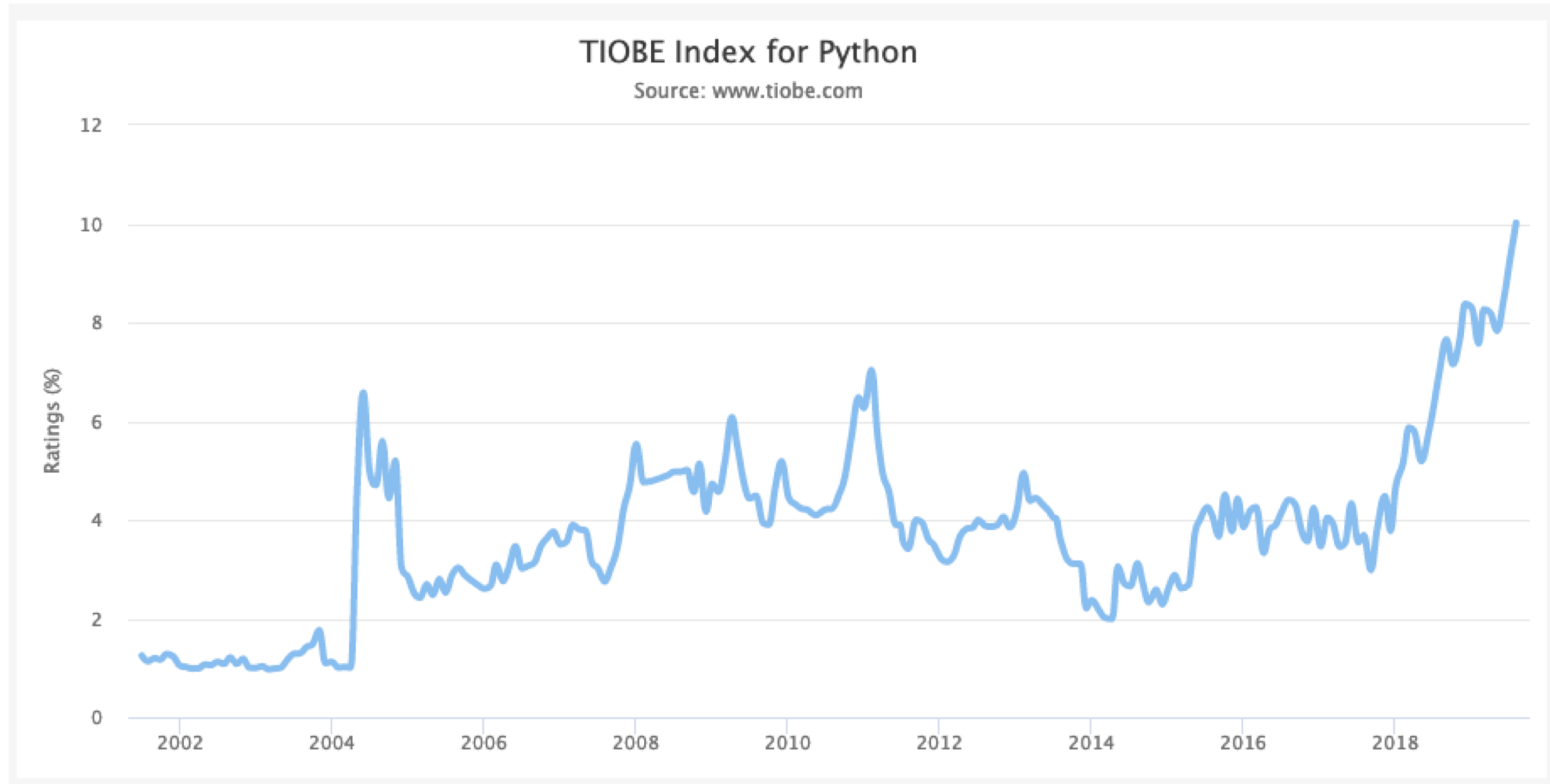
Propiedades del lenguaje Python

- Lenguaje de propósito general
- Multiparadigma
- Lenguaje interpretado
- Multiplataforma
- Tipado dinámico
- Lenguaje limpio

TIOBE index

Aug 2019	Aug 2018	Programming Language	Change
1	1	Java	-0.85%
2	2	C	+0.19%
3	4	Python	+3.03%
4	3	C++	-1.41%
5	6	C#	+0.30%
6	5	Visual Basic .NET	-1.07%
7	8	JavaScript	-0.15%
8	7	PHP	-0.85%
9	14	Objective-C	+0.33%
10	9	SQL	-0.69%

TIOBE Index



Estructura de un programa

Conceptos básicos. Sentencias

- **Asignación**
 - Evaluar una expresión y almacenar el resultado en una variable
 - $a = 3 + 4$
- **Entrada/Salida**
 - Transferencia de información con el exterior a través de dispositivos de entrada (teclado, ficheros ...) y salida (monitor, impresora ...)
 - print, input, ...
- **Control de flujo**
 - Selección
 - Tomar una decisión para seguir un camino u otro
 - if, elif, else
 - Repetición
 - Repetir la ejecución de una o varias de sentencias
 - while, for
- **Subrutinas**
 - Secuencia de instrucciones que se ejecutan de manera ordenada al ser llamadas

Conceptos básicos. Elementos

- Alfabeto: conjunto de símbolos válidos para construir textos en el lenguaje:
 - Palabras clave: {if, elif, else, while ...}
 - Dígitos: {0,...,9}
 - Caracteres: {'a', ..., 'z', 'A', ..., 'Z', '#', '?' ...}
- Léxico: vocabulario formado a partir del alfabeto
- Sintaxis: conjunto de reglas gramaticales que permiten derivar frases correctas en el lenguaje
- Semántica: conjunto de reglas que permiten derivar frases (*sentencias*) con sentido (significado) en el lenguaje.

Conceptos básicos

- Un programa se crea utilizando un alfabeto de símbolos (léxico) para construir:
 - Identificadores
 - Etiquetas
 - Comentarios
 - Directivas
 - Constantes
 - Variables
 - Números
 - Cadenas de caracteres

Elementos de un programa

Símbolos predefinidos

Conjunto de símbolos que constituyen el alfabeto del lenguaje

- En el caso de Python3 será el conjunto de caracteres Unicode
- En el caso de Python2 será el conjunto de caracteres ASCII

Python 2

```
>>> len('piña')
```

5

Python 3

```
>>> len('piña')
```

4

Símbolos predefinidos

- Los caracteres ingleses y ASCII (128 caracteres)

	0	1	2	3	4	5	6	7	8	9
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	np	cr	so	si	dle	dc1	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	"	#	\$	%	&	'
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	¿	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{		}	~	del		

Símbolos predefinidos

- La tabla ASCII tiene solamente 128 caracteres que contienen
 - Las letras inglesas a-z, A-Z
 - Los dígitos 0-9
 - Los símbolos de puntuación ingleses y el espacio
 - Códigos de control que tienen su origen en máquinas de teletipo (obsoletos hoy en día)
- El ASCII es incapaz de representar caracteres que se escapan de esta tabla y no puede representar ciertos caracteres, como los caracteres acentuados en europa.
- Unicode es un estándar que abarca todos los caracteres de las ortografías del mundo. Además tiene espacio para crecer (puede representar más de un millón de caracteres aunque actualmente solo están definidos unos 110000)

Símbolos especiales

Son símbolos con un significado especial dentro del lenguaje que se usan para representar delimitadores, operadores y otros elementos sintácticos.

Ejemplo:

=	-	*	/	%	=	<	>
{	}	&		,	:	^	**
[	]	(	)	>=	<=	!=	.
>>	<<	=	@	?	`	\$	.

Lista completa: https://docs.python.org/3/reference/lexical_analysis.html

Palabras reservadas

- Las palabras reservadas (keywords) se utilizan para especificar nombres de sentencias, tipos de datos, operadores...
 - No se pueden redefinir para usarlas como identificadores

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Identificadores

Son nombres que pueden utilizarse para denominar una función, una constante, una variable u otros elementos:

identificador: xid_start xid_continue*

xid_start: conjunto de letras + underscore (_)

xid_continue: xid_start + dígitos

El nombre debe ser apropiado: tener un significado adecuado al uso destinado para el identificador.

Python es case-sensitive. El identificador Edad es distinto de edad.

Comentarios

- Es cualquier secuencia de caracteres y que no es ejecutado por la máquina y que sirve para documentar los programas.
- En Python cualquier cadena de caracteres que aparece después de # es considerado un comentario

```
>>> # comentario de una línea en python
```

```
...
```

```
>>> 3+4 # esto es una suma
```

```
7
```

Constantes

Identificadores que se corresponden a un valor constante que no varía durante la ejecución del programa.

En el lenguaje Python no existen las constantes, aún así por convención las variables que pertenecen a un módulo se tratan como si fueran constantes.

Una convención muy habitual para los identificadores de las constantes es emplear letras mayúsculas.

Ejemplo de la identificadores:

PI = 3.141592653589793

EDAD_JUBILACION = 67

```
>>>import math
>>>math.pi
3.141592653589793
>>>math.pi = 3
3
```

Si el math.pi se comportase como una constante no se podría modificar su valor

Tipos de datos

- En Python los números pueden ser enteros, reales o complejos

- Enteros

```
>>> type(3)
<class 'int'>
```

- Reales

```
>>> type(3.0)
<class 'float'>
```

- Complejos

```
>>> type(4+3j)
<class 'complex'>
```

- En Python el texto se guarda como una cadena de caracteres.

```
>>> type("foobar")
<class 'str'>
```

Variables

- Los valores de una variable pueden cambiar durante la ejecución de un programa.
- Se definen como:
 nombre_variable = valor
- Ejemplo

```
>>> edad=18
>>> type(edad)
<class 'int'>
>>> edad
18
>>> edad = 20
```

Variables

- Algunos lenguajes de programación implementan tipado estático de variables, lo que quiere decir que una variable solo podrá mantener un tipo de dato.
- Sin embargo, Python es de tipado dinámico, lo que indica que una variable puede cambiar de tipo de dato

```
>>> valor=3
>>> type(valor)
<class 'int'>
>>> valor=3.5
>>> type(valor)
<class 'float'>
>>> valor
3.5
>>> valor='aceptado'
>>> type(valor)
<class 'str'>
```

Salida y entrada

Salida y entrada

Python es un lenguaje interpretado y algunos interpretes ejecutan las instrucciones según se van introduciendo (por ejemplo el comando python de la terminal). Estos interpretes muestran el valor de las variables y de las expresiones.

```
>>> 3
```

```
3
```

```
>>> a = 3
```

```
>>> a
```

```
3
```

```
>>> 3+4
```

```
7
```

Salida y entrada

Aunque algunos interpretes tienen el comportamiento de mostrar el valor de las variables y expresiones esto no sucede con todos los interpretes y no pasa cuando se ejecuta el código Python de un archivo.

Archivo ej.py

```
3  
a = 3  
3+4
```

```
$ python ej.py  
$
```

Salida. print

- Para mostrar información con python se puede usar la salida estándar y para ello se utiliza la función `print` de python.

```
>>> a=3
>>> print(a)
3
```

- La información a mostrar se puede personalizar. La función `print` permite la introducción de varios parámetros que se mostraran por pantalla de manera ordenada

```
>>> print('El valor de a es ', a, '.')
El valor de a es 3 .
```

Secuencias de "escape"

Existen unos ciertas secuencias dentro de las cadenas de caracteres que se consideran secuencias de escapa que tienen un significado especial

`\n` salto de línea

`\t` tabulador

```
>>> print('Item\tPrecio\nAgua\t1.0\nCafe\t2.5\nTotal\t3.5')
```

```
Item    Precio
```

```
Agua    1.0
```

```
Cafe    2.5
```

```
Total   3.5
```

Secuencias de "escape"

Las secuencias de escapa empiezan con el carácter \ y con el se pueden mostrar valores como el propio \ o las comillas simples y dobles

```
>>> print('Backslash \\'.')
```

```
Backslash \.
```

```
>>> print('Single quote\''.')
```

```
Single quote'.
```

```
>>> print('Double quote\".')
```

```
Double quote".
```

Entrada

Para poder hacer programas interactivos es necesario que el usuario introduzca los valores. La función `input` permite esa acción:

```
>>> nombre = input('Introduce tu nombre: ')
```

```
Introduce tu nombre: Fran
```

```
>>> print('Hola', nombre)
```

```
Hola Fran
```

Tipos de datos y operadores

Tipos de datos y operadores. bool

Es un tipo de dato binario que solo puede tener dos valores: True y False.

```
>>> type (True)
<class 'bool'>
>>> type (False)
<class 'bool'>
```

Tipos de datos y operadores. bool

El tipo de datos bool es la respuesta de operaciones lógicas

```
>>> 3 == 1+2
```

```
True
```

```
>>> 3 == 1+3
```

```
False
```

Tipos de datos y operadores. bool

Existen operaciones específicas para los datos bool

```
>>> not True
```

```
False
```

```
>>> True or False
```

```
True
```

```
>>> True and False
```

```
False
```

Tipos de datos y operadores. bool

or	False	True
False	False	True
True	True	True

and	False	True
False	False	False
True	False	True

!=	False	True
False	False	False
True	True	False

not	False	True
	True	False

Tipos de datos y operadores. Expresiones de comparación

<	Menor que	!=	Desigualdad
<=	Menor o igual que	is	es
>=	Mayor o igual que	is not	No es
>	Mayor que	in	Está contenido
==	igualdad	not in	No está contenido

```
>>> 4 >= 3
```

```
True
```

```
>>> 4 <= 3
```

```
False
```

```
>>> 3 == 4
```

```
False
```

```
>>> 4 != 4
```

```
False
```

Tipos de datos y operadores. Operadores aritméticos

Python incluye un conjunto completo de operadores aritméticos, incluyendo los operadores binarios, para las cuatro operaciones matemáticas básicas: + suma, - resta, * multiplicación y / división

```
>>> 4+7
```

```
11
```

```
>>> 4-7
```

```
-3
```

```
>>> 4*7
```

```
28
```

Tipos de datos y operadores. Operadores aritméticos

El símbolo menos puede ser usado como un operador binario (resta) o como operador unario.

```
>>> -4  
-4
```

Mientras que las operaciones de +, - y * no pueden devolver float como int, la división siempre devuelve el tipo de dato float

```
>>> 4/7  
0.5714285714285714  
>>> 12/4  
3.0  
>>> type(4/7)  
<class 'float'>  
>>> type(12/4)  
<class 'float'>
```

Tipos de datos y operadores. Operadores aritméticos

En Python se puede usar los operadores de asignación aumentados `+=`, `-=`, `*=` y `/=` que permiten usar una notación comprimida

Operación	Notación comprimida	Notación sin comprimir
Suma	<code>a += 5</code>	<code>a = a + 5</code>
Resta	<code>b -= 5</code>	<code>b = b - 5</code>
Multiplicación	<code>c *= 5</code>	<code>c = c * 5</code>
División	<code>d /= 5</code>	<code>d = d / 5</code>

```
>>> a = b = c = d = 13
```

```
>>> a += 5
```

```
>>> b -= 5
```

```
>>> c *= 5
```

```
>>> d /= 5
```

```
>>> print('a = {}, b = {}, c = {}, d = {}'.format(a,b,c,d))
... .format(a,b,c,d))
a = 18, b = 8, c = 65, d = 2.6
```

Tipos de datos y operadores. Operadores aritméticos

Además de los operadores aritméticos básicos, Python cuenta con otros operadores como el exponente (**), división entera (//) y módulo (%)

```
>>> 2 ** 3
```

```
8
```

```
>>> 5 // 2
```

```
2
```

```
>>> type(5 // 2)
```

```
<class 'int'>
```

```
>>> 5 % 2
```

```
1
```

Orden de las operaciones

Cuando en una expresión aparece más de un operador, el orden de evaluación depende de las reglas de precedencia. Para los operadores matemáticos, Python sigue las convenciones matemáticas.

1. Paréntesis
2. Exponente: `**`
3. Negación: `-`
4. Multiplicación, División, División entera, Modulo: `*`, `/`, `//`, `%`
5. Suma, Resta: `+`, `-`

Orden de las operaciones

Ejemplos:

```
>>> 4*2+3
```

```
11
```

```
>>> 4*2**3
```

```
32
```

```
>>> -2*4+3
```

```
-5
```

```
>>> -2**4
```

```
-16
```

```
>>> 14/3/2
```

```
2.33333333333333335
```

```
>>> 4*(2+3)
```

```
20
```

```
>>> (4*2)**3
```

```
512
```

```
>>> -(2*4+3)
```

```
-11
```

```
>>> (-2)**4
```

```
16
```

```
>>> 14/(3/2)
```

```
9.3333333333333334
```

Depuración de programas

Depuración de programas

En ocasiones, los programas pueden contener algunos errores, como no escribir las expresiones de manera completa o realizar operaciones como la división de un número entre cero.

El lenguaje de Python nos devolverá un error indicándonos el tipo de fallo que se ha encontrado

```
>>> a = 4 +  
File "<stdin>", line 1  
    a = 4 +  
        ^  
SyntaxError: invalid syntax
```

```
>>> cadena = 'adfasdf  
File "<stdin>", line 1  
    cadena = 'adfasdf  
                ^  
SyntaxError: EOL while scanning string literal
```

```
>>> 7/0  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ZeroDivisionError: division by zero
```

```
>>> 4 + 'hola'  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s)  
for +: 'int' and 'str'
```